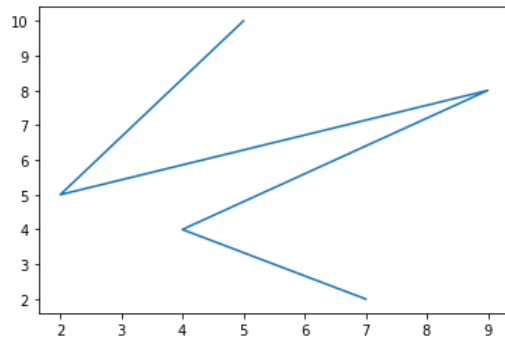


```
In [ ]: ##### MACHINE LEARNING LAB WEEK-2 #####
```

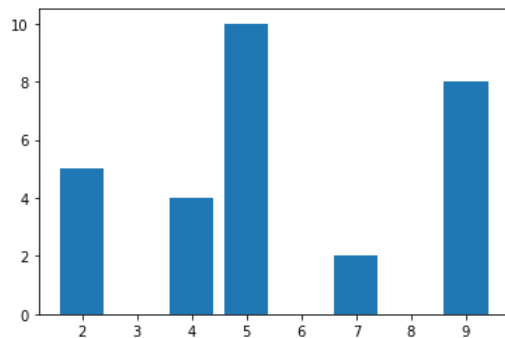
```
In [ ]: # LINE PLOT
```

```
In [3]: # importing matplotlib module
import matplotlib.pyplot as plt
from matplotlib import pyplot as plt
# x-axis values
x = [5, 2, 9, 4, 7]
# Y-axis values
y = [10, 5, 8, 4, 2]
# Function to plot
plt.plot(x,y)
# function to show the plot
plt.show()
```



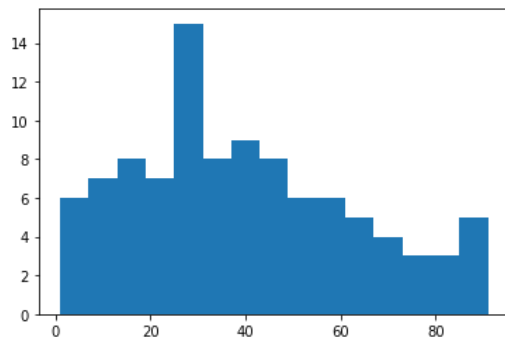
```
In [ ]: # BAR PLOT
```

```
In [4]: # importing matplotlib module
from matplotlib import pyplot as plt
# x-axis values
x = [5, 2, 9, 4, 7]
# Y-axis values
y = [10, 5, 8, 4, 2]
# Function to plot the bar
plt.bar(x,y)
# function to show the plot
plt.show()
```

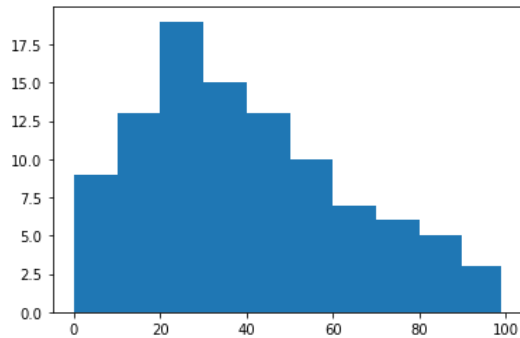


```
In [ ]: # HIST PLOT
```

```
In [1]: import matplotlib.pyplot as plt
x = [1, 1, 2, 3, 3, 5, 7, 8, 9, 10,
10, 11, 11, 13, 13, 15, 16, 17, 18, 18,
18, 19, 20, 21, 21, 23, 24, 24, 25, 25,
25, 25, 26, 26, 26, 27, 27, 27, 27, 27,
29, 30, 30, 31, 33, 34, 34, 34, 35, 36,
36, 37, 37, 38, 38, 39, 40, 41, 41, 42,
43, 44, 45, 45, 46, 47, 48, 48, 49, 50,
51, 52, 53, 54, 55, 55, 56, 57, 58, 60,
61, 63, 64, 65, 66, 68, 70, 71, 72, 74,
75, 77, 81, 83, 84, 87, 89, 90, 90, 91]
plt.hist(x, bins=15)
plt.show()
```



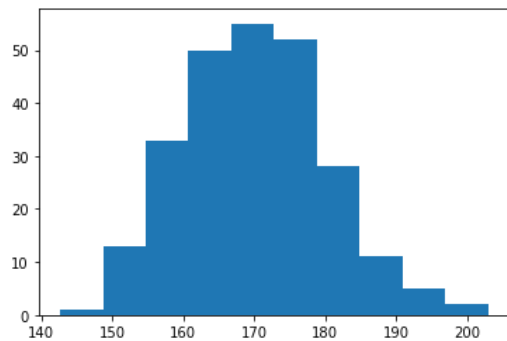
```
In [6]: import matplotlib.pyplot as plt
x = [1, 1, 2, 3, 3, 5, 7, 8, 9, 10,
10, 11, 11, 13, 13, 15, 16, 17, 18, 18,
18, 19, 20, 21, 21, 23, 24, 24, 25, 25,
25, 25, 26, 26, 26, 27, 27, 27, 27, 27,
29, 30, 30, 31, 33, 34, 34, 34, 35, 36,
36, 37, 37, 38, 38, 39, 40, 41, 41, 42,
43, 44, 45, 45, 46, 47, 48, 48, 49, 50,
51, 52, 53, 54, 55, 55, 56, 57, 58, 60,
61, 63, 64, 65, 66, 68, 70, 71, 72, 74,
75, 77, 81, 83, 84, 87, 89, 90, 90, 91
]
plt.hist(x, bins=[0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 99])
plt.show()
```



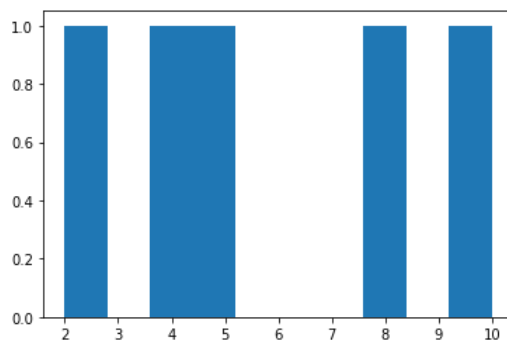
```
In [12]: # Example 3
# For simplicity we use NumPy to randomly generate an array with 250 values,
# where the values will concentrate around 170, and the standard deviation is 10.

import matplotlib.pyplot as plt
import numpy as np
x = np.random.normal(170, 10, 250)
print(x)
plt.hist(x)
plt.show()
```

```
[162.87804264 179.9645458 168.40573171 202.93465659 160.32949528
167.94074894 162.29320046 184.67776934 180.04071414 157.08448467
174.46795003 167.02000136 177.7725798 167.64654088 191.8406639
173.56191416 185.43787195 177.98854672 172.61484461 181.88419377
173.74606544 163.24825075 188.9696771 158.7149624 170.73230806
162.64687944 175.17103766 154.95463809 161.02543846 173.08237921
169.15422753 167.25065092 165.71592192 182.28729998 161.14374288
182.30116143 172.49386566 175.22881511 186.54898228 175.59330037
184.592481 159.20927398 158.47969331 163.22562692 191.85260615
170.1940481 158.40812921 163.62246669 149.61817424 180.46945309
175.14563587 174.2257285 169.15846021 170.56484345 153.87524224
184.04458975 177.9104087 158.81913363 187.00517463 165.99351352
176.86003029 171.27893675 171.0572424 154.76511109 155.35021644
165.01211263 157.43386201 172.81434276 177.06517749 162.18990565
184.66624459 170.10900551 153.46893631 181.24360561 174.01213693
174.79771438 173.66365237 171.65501874 172.06256501 150.65658772
150.86533458 178.3582489 174.74134006 165.22345638 161.60264508
172.01744712 158.40528938 162.39739909 152.07299486 185.94805614
176.39253122 176.15990114 155.07645353 161.02409743 166.60284254
177.80924039 195.8076951 151.87045146 166.98362374 170.17479777
175.22206746 180.30916851 160.89778749 189.96804622 151.99565949
164.02342325 199.96992412 181.66574933 162.3352502 161.96517426
173.45229693 169.74524549 164.31650373 172.1079246 163.09411084
161.15653775 187.54866368 170.22040254 164.19414545 178.56831721
169.79686944 157.66185604 163.20776288 180.97742849 164.95306077
176.19643755 177.02768159 158.81687813 178.88518917 170.01599797
165.79707873 182.33024928 168.89820347 164.73031558 162.3709165
162.80107381 166.86585198 173.30702785 164.72338679 172.54431916
156.24867787 142.68645765 177.78912968 177.73946343 177.87637617
169.35592753 166.75443197 181.53176764 171.18876224 185.8879688
166.43532454 169.54331009 174.77898349 160.50289736 179.29095348
174.97517117 158.10999879 158.06395795 169.37057686 180.56117434
167.13874565 159.06308664 170.72914921 180.74216642 173.92215616
164.15730932 157.70516436 178.58593192 164.36717426 166.7235902
161.9187714 165.62565982 167.42217916 177.20508881 185.77680736
165.13950181 167.81770873 171.27493985 174.59717076 171.14510039
174.25429476 168.04314591 164.21280164 176.07775911 182.43443955
170.37694336 159.82239282 176.58087527 176.29819304 172.7352194
158.34373884 165.06060225 164.40863753 170.44363049 154.03519015
174.8692276 181.02707687 195.31525078 172.97196003 153.61473203
180.11151111 162.59474603 178.25904553 167.06724726 154.46854902
169.45256299 158.71021494 167.47023168 161.93507145 195.62906185
175.44673283 178.18172941 179.91190612 158.53540438 182.00210227
169.30850425 149.75489442 188.22870153 169.33732771 167.68229414
174.03218216 169.65750757 167.75761173 184.30932418 160.37099747
173.32270446 179.34229382 171.35667571 161.01600113 158.89719542
155.36038239 174.91800766 164.31443941 174.75833329 165.81047544
187.01795152 167.23304111 156.0558465 167.27350281 175.11045154
155.05622079 162.53735871 155.72523541 167.82258512 153.87536808
170.24522617 160.47334824 166.71603008 183.1826432 160.49232259]
```



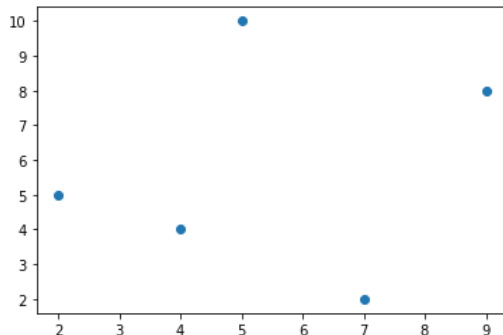
```
In [13]: # Example 4
# importing matplotlib module
from matplotlib import pyplot as plt
# Y-axis values
y = [10, 5, 8, 4, 2]
# Function to plot histogram
plt.hist(y)
# Function to show the plot
plt.show()
```



```
In [14]: # 4. Scatter plot:
# A scatter plot is a diagram where each value in the data set is represented by a dot.
# The Matplotlib module has a method for drawing scatter plots, it needs two arrays of the
# same length, one for the values of the x-axis, and one for the values of the y-axis.

# Example 5

# importing matplotlib module
from matplotlib import pyplot as plt
# x-axis values
x = [5, 2, 9, 4, 7]
# Y-axis values
y = [10, 5, 8, 4, 2]
# Function to plot scatter
plt.scatter(x, y)
# function to show the plot
plt.show()
```



```
In [ ]: # SciPy ==> Scientific Python.
```

```
In [15]: from scipy import constants
print(constants.peta) #1000000000000000.0
print(constants.tera) #1000000000000.0
print(constants.giga) #1000000000.0
print(constants.mega) #1000000.0
print(constants.kilo) #1000.0
print(constants.hecto) #100.0
print(constants.deka) #10.0
print(constants.deci) #0.1
print(constants.cent) #0.01
print(constants.milli) #0.001
print(constants.micro) #1e-06
print(constants.nano) #1e-09
print(constants.pico) #1e-12
```

```
1000000000000000.0
1000000000000000.0
1000000000000.0
1000000.0
1000.0
100.0
10.0
0.1
0.01
0.001
1e-06
1e-09
1e-12
```

```
In [ ]: # Sparse Data
# Sparse data is data that has mostly unused elements (elements that don't carry any information).
# It can be an array like this one: [1, 0, 2, 0, 0, 3, 0, 0, 0, 0, 0]
# Sparse Data: is a data set where most of the item values are zero.
# Dense Array: is the opposite of a sparse array: most of the values are not zero.
# In scientific computing, when we are dealing with partial derivatives in linear algebra we will
# come across sparse data.
```

```
"""
Work With Sparse Data
SciPy has a module, scipy.sparse that provides functions to deal with sparse data.
There are primarily two types of sparse matrices that we use:
CSC - Compressed Sparse Column. For efficient arithmetic, fast column slicing.
CSR - Compressed Sparse Row. For fast row slicing, faster matrix vector products
We will use the CSR matrix.
CSR Matrix
We can create CSR matrix by passing an array into function scipy.sparse.csr_matrix().
"""
```

```
In [17]: # Example 7
# Create a CSR matrix from an array:
import numpy as np
from scipy.sparse import csr_matrix
arr = np.array([0, 0, 0, 0, 0, 0, 1, 1, 0, 2])
```

```
print(arr)
print(csr_matrix(arr))
```

```
[0 0 0 0 0 1 1 0 2]
(0, 5)      1
(0, 6)      1
(0, 8)      2
```

In [18]: *# Sparse Matrix Methods*
Viewing stored data (not the zero items) with the data property:
Example 8

```
import numpy as np
from scipy.sparse import csr_matrix
arr = np.array([[0, 0, 0], [0, 0, 1], [1, 0, 2]])
print(csr_matrix(arr).data)
```

```
[1 1 2]
```

In [19]: *# Converting from csr to csc with the tocsc() method:*
Example 9

```
import numpy as np
from scipy.sparse import csr_matrix
arr = np.array([[0, 0, 0], [0, 0, 1], [1, 0, 2]])
newarr = csr_matrix(arr).tocsc()
print(newarr)
```

```
(2, 0)      1
(1, 2)      1
(2, 2)      2
```

In []: *"""*
Pandas
Pandas is a Python library used for working with data sets.
It has functions for analyzing, cleaning, exploring, and manipulating data.
The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis"
and was created by Wes McKinney in 2008.
"""

In [21]: *import pandas*
mydataset = { 'cars': ["BMW", "Volvo", "Ford"],
'passings': [3, 7, 2] }
myvar = pandas.DataFrame(mydataset)
print(myvar)

```
cars  passings
0    BMW        3
1  Volvo        7
2   Ford        2
```

In [22]: *# Create an alias with the as keyword while importing:*
import pandas as pd
Now the Pandas package can be referred to as pd instead of pandas.
Example 11

```
import pandas as pd
mydataset = {
    'cars': ["BMW", "Volvo", "Ford"],
    'passings': [3, 7, 2]
}
myvar = pd.DataFrame(mydataset)
print(myvar)
```

```
cars  passings
0    BMW        3
1  Volvo        7
2   Ford        2
```

In [23]: *# Pandas Series:*
A Pandas Series is like a column in a table.

```
import pandas as pd
new_series = pd.Series([5,6,7,8,9,10])
print(new_series)
```

```
0    5
1    6
2    7
3    8
4    9
5   10
dtype: int64
```

In [2]: *# A series has ways to extract all of the values in the series,*
as well as individual elements by index

Example 13

```
import pandas as pd
new_series = pd.Series([5,6,7,8,9,10])
print(new_series.values)
```

```
print('_____')
print(new_series[2])
```

```
[ 5  6  7  8  9 10]
```

```
_____
```

```
7
```

In [25]: *# You can also provide an index manually using index argument.
When you have created Labels, you can access an item by referring to the Label.*

Example 14

```
import pandas as pd
new_series= pd.Series([5,6,7,8,9,10], index=['a','b','c','d','e','f'])
print(new_series)
print('_____')
print(new_series.values)
print('_____')
print(new_series['f'])
```

```
a    5
b    6
c    7
d    8
e    9
f   10
```

```
dtype: int64
```

```
[ 5  6  7  8  9 10]
```

```
_____
```

```
10
```

In [26]: *# It is easy to retrieve several elements of a series by their indices or make group assignments
Example 15*

```
import pandas as pd
new_series= pd.Series([5,6,7,8,9,10], index=['a','b','c','d','e','f'])
print(new_series)
print('_____')
print(new_series[['b', 'c', 'f']])
```

```
a    5
b    6
c    7
d    8
e    9
f   10
```

```
dtype: int64
```

```
b    6
c    7
f   10
```

```
dtype: int64
```

In [4]: *# Filtering and maths operations are easy with Pandas as well.*

Example 16

```
import pandas as pd
new_series= pd.Series([5,6,7,8,9,10], index=['a','b','c','d','e','f'])
new_series2= new_series[new_series>7] # check with other numbers instead of 0
print(new_series2)
print('_____')
new_series2= new_series[new_series>0] * 2
print(new_series2)
```

```
d    8
e    9
f   10
```

```
dtype: int64
```

```
a   10
b   12
c   14
d   16
e   18
f   20
```

```
dtype: int64
```

In [30]: *"""
DataFrame:
A Pandas DataFrame is a 2 dimensional data structure, like a 2 dimensional array,
or a table with rows and columns.
Series is like a column, a DataFrame is the whole table.
Simplistically, a data frame is a table, with rows and columns.
Each column in a data frame is a series object.
Rows consist of elements inside series.
"""*

Example 17

#Create a simple Pandas DataFrame:

```
import pandas as pd
```

```
data = {
    "calories": [420, 380, 390],
    "duration": [50, 40, 45]
}
#Load data into a DataFrame object:
df = pd.DataFrame(data)
print(df)
```

```
   calories  duration
0        420         50
1        380         40
2        390         45
```

In [33]: *# Locate Row*
As you can see from the result above, the DataFrame is like a table with rows and columns.
Pandas use the loc attribute to return one or more specified row(s)
Example 18

Return row 0:
Refer to the row index:
print(df.loc[0])

```
calories    420
duration     50
Name: 0, dtype: int64
```

In [34]: *# Example 19*
Add a List of names to give each row a name:

```
import pandas as pd
data = {
    "calories": [420, 380, 390],
    "duration": [50, 40, 45]
}
df = pd.DataFrame(data, index = ["day1", "day2", "day3"])
print(df)
```

```
   calories  duration
day1      420         50
day2      380         40
day3      390         45
```

In [35]: *# You can also create a data frame from a list.*
Example 20

```
import pandas as pd
# create a Pandas DataFrame
list2=[[0,1,2],[3,4,5],[6,7,8]]
df= pd.DataFrame(list2)
print("DataFrame:")
print(df)
# set the column labels
df.columns = ['C1','C2','C3']
print("Column names are Updated:")
print(df)
```

```
DataFrame:
   0  1  2
0  0  1  2
1  3  4  5
2  6  7  8
Column names are Updated:
   C1  C2  C3
0   0   1   2
1   3   4   5
2   6   7   8
```

In [36]: *# Example 21*
import pandas as pd
df=pd.DataFrame({
 'Country': ['Kazakhstan', 'Russia', 'Belarus', 'Ukraine'],
 'Population': [17.04,143.5,9.5,45.5],
 'Square':[2724902, 17125191,207600,603628]
})
print(df)

```
   Country  Population  Square
0  Kazakhstan      17.04  2724902
1    Russia      143.50  17125191
2   Belarus       9.50   207600
3   Ukraine      45.50   603628
```

In [41]: *# Example 22*

```
import pandas as pd
df=pd.DataFrame({
    'Country': ['Kazakhstan', 'Russia', 'Belarus', 'Ukraine'],
    'Population': [17.04,143.5,9.5,45.5],
    'Square':[2724902, 17125191,207600,603628]
})
print(df.columns)
```

```
print('_____')
print(df.index)
```

```
Index(['Country', 'Population', 'Square'], dtype='object')
```

```
RangeIndex(start=0, stop=4, step=1)
```

In [43]: *# You could provide an index when creating a data frame*

Example 23

```
import pandas as pd
df=pd.DataFrame({
'Country': ['Kazakhstan','Russia','Belarus','Ukraine'],
'Population': [17.04,143.5,9.5,45.5],
'Square':[2724902,17125191,207600,603628],
}, index = ['KZ','RU','BY','UA'])
print(df)
```

	Country	Population	Square
KZ	Kazakhstan	17.04	2724902
RU	Russia	143.50	17125191
BY	Belarus	9.50	207600
UA	Ukraine	45.50	603628

In [5]: *# Load the CSV into a DataFrame*

```
import pandas as pd
df = pd.read_csv('mtcars.csv')
print(df.to_string()) # use to_string() to print the entire DataFrame.
print(df) #Print the DataFrame without the to_string() method
```

	model	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
0	Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
1	Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
2	Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
3	Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
4	Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
5	Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
6	Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
7	Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
8	Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
9	Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
10	Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
11	Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3
12	Merc 450SL	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3
13	Merc 450SLC	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	3
14	Cadillac Fleetwood	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	4
15	Lincoln Continental	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	4
16	Chrysler Imperial	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
17	Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
18	Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
19	Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
20	Toyota Corona	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1
21	Dodge Challenger	15.5	8	318.0	150	2.76	3.520	16.87	0	0	3	2
22	AMC Javelin	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	2
23	Camaro Z28	13.3	8	350.0	245	3.73	3.840	15.41	0	0	3	4
24	Pontiac Firebird	19.2	8	400.0	175	3.08	3.845	17.05	0	0	3	2
25	Fiat X1-9	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4	1
26	Porsche 914-2	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	2
27	Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5	2
28	Ford Pantera L	15.8	8	351.0	264	4.22	3.170	14.50	0	1	5	4
29	Ferrari Dino	19.7	6	145.0	175	3.62	2.770	15.50	0	1	5	6
30	Maserati Bora	15.0	8	301.0	335	3.54	3.570	14.60	0	1	5	8
31	Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4	2

	model	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	\
0	Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	
1	Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	
2	Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	
3	Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	
4	Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	
5	Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	
6	Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	
7	Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	
8	Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	
9	Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	
10	Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	
11	Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	
12	Merc 450SL	17.3	8	275.8	180	3.07	3.730	17.60	0	0	
13	Merc 450SLC	15.2	8	275.8	180	3.07	3.780	18.00	0	0	
14	Cadillac Fleetwood	10.4	8	472.0	205	2.93	5.250	17.98	0	0	
15	Lincoln Continental	10.4	8	460.0	215	3.00	5.424	17.82	0	0	
16	Chrysler Imperial	14.7	8	440.0	230	3.23	5.345	17.42	0	0	
17	Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	
18	Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	
19	Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	
20	Toyota Corona	21.5	4	120.1	97	3.70	2.465	20.01	1	0	
21	Dodge Challenger	15.5	8	318.0	150	2.76	3.520	16.87	0	0	
22	AMC Javelin	15.2	8	304.0	150	3.15	3.435	17.30	0	0	
23	Camaro Z28	13.3	8	350.0	245	3.73	3.840	15.41	0	0	
24	Pontiac Firebird	19.2	8	400.0	175	3.08	3.845	17.05	0	0	
25	Fiat X1-9	27.3	4	79.0	66	4.08	1.935	18.90	1	1	
26	Porsche 914-2	26.0	4	120.3	91	4.43	2.140	16.70	0	1	
27	Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.90	1	1	
28	Ford Pantera L	15.8	8	351.0	264	4.22	3.170	14.50	0	1	
29	Ferrari Dino	19.7	6	145.0	175	3.62	2.770	15.50	0	1	
30	Maserati Bora	15.0	8	301.0	335	3.54	3.570	14.60	0	1	
31	Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.60	1	1	

	gear	carb
0	4	4
1	4	4
2	4	1
3	3	1
4	3	2
5	3	1
6	3	4
7	4	2
8	4	2
9	4	4
10	4	4
11	3	3
12	3	3
13	3	3
14	3	4
15	3	4
16	3	4
17	4	1
18	4	2
19	4	1
20	3	1
21	3	2
22	3	2
23	3	4

24	3	2
25	4	1
26	5	2
27	5	2
28	5	4
29	5	6
30	5	8
31	4	2

In []:

In []:

In []:

In []:

In []:

```
In [1]: import numpy as np
from scipy.sparse import csr_matrix
arr = np.array([0, 0, 0, 0, 0, 1, 1, 0, 2])
print(csr_matrix(arr))
```

```
(0, 5)      1
(0, 6)      1
(0, 8)      2
```